

On This Page >

Authoring a 'clan.service' module

Service Module Specification

This section explains how to author a clan service module. We discussed the initial architecture in [01-clan-service-modules](#) and decided to rework the format.

For the full specification and current state see: [Service Author Reference](#)

A Minimal module

First of all we need to register our module into the `clan.modules` attribute. Make sure to choose a unique name so the module doesn't have a name collision with any of the core modules.

While not required, we recommend to prefix your module attribute name.

If you export the module from your flake, other people will be able to import it and use it within their clan.

i.e. `@hsjobeki/customNetworking`

flake.nix

```
1 # ...
2 outputs = inputs: inputs.flake-parts.lib.mkFlake { inherit inputs; } ({
3   imports = [ inputs.clan-core.flakeModules.default ];
4   # ...
5   # Sometimes this attribute set is defined in clan.nix
6   clan = {
7     # If needed: Exporting the module for other people
8     modules."@hsjobeki/customNetworking" = import ./service-modules/;
9     # We could also inline the complete module spec here
```

```

10      # For example
11      # {...}: { _class = "clan.service"; ... };
12  };
13 })

```

The imported module file must fulfill at least the following requirements:

- Be an actual module. That means: Be either an attribute set; or a function that returns an attribute set.
- Required `_class = "clan.service"`
- Required `manifest.name = "<name of the provided service>"`

/service-modules/networking.nix

```

1 {
2     _class = "clan.service";
3     manifest.name = "zerotier-networking";
4     # ...
5 }

```

For more attributes see: [Service Author Reference](#)

Adding functionality to the module

While the very minimal module is valid in itself, it has no way of adding any machines to it, because it doesn't specify any roles.

The next logical step is to think about the interactions between the machines and define *roles* for them.

Here is a short guide with some conventions:

- ☐ If they all have the same relation to each other `peer` is commonly used. `peers` can often talk to each other directly.
- ☐ Often machines don't necessarily have direct relation to each other and there is one elevated machine in the middle classically known as `client-server`. `clients` are less likely to talk directly to each other than `peers`
- ☐ If your machines don't have any relation and/or interactions to each other you should reconsider if the desired functionality is really a multi-host service.

/service-modules/networking.nix

```

1 {
2   _class = "clan.service";
3   manifest.name = "zerotier-networking";
4
5   # Define what roles exist
6   roles.peer = {};
7   roles.controller = {};
8   # ...
9 }

```

Next we need to define the settings and the behavior of these distinct roles.

/service-modules/networking.nix

```

1 {
2   _class = "clan.service";
3   manifest.name = "zerotier-networking";
4
5   # Define what roles exist
6   roles.peer = {
7     interface = {
8       # These options can be set via 'roles.client.settings'
9       options.ipRanges = mkOption { type = listOf str; };
10    };
11
12    # Maps over all instances and produces one result per instance.
13    perInstance = { instanceName, settings, machine, roles, ... }: {
14      # Analog to 'perSystem' of flake-parts.
15      # For every instance of this service we will add a nixosModule
16      nixosModule = { config, ... }: {
17        # Interaction examples what you could do here:
18        # - Get some settings of this machine
19        # settings.ipRanges
20        #
21        # - Get all controller names:
22        # allControllerNames = lib.attrNames roles.controller.machines;
23        #
24        # - Get all roles of the machine:
25        # machine.roles
26        #
27        # - Get the settings that were applied to a specific controller
28        # roles.controller.machines.json.settings
29        #
30        # Add one systemd service for every instance

```

```

31         systemd.services.zerotier-client-${instanceName} = {
32             # ... depend on the '.config' and 'perInstance arguments
33         };
34     };
35 }
36 };
37 roles.controller = {
38     interface = {
39         # These options can be set via 'roles.server.settings'
40         options.dynamicIp.enable = mkOption { type = bool; };
41     };
42     perInstance = { ... }: {};
43 };
44
45 # Maps over all machines and produces one result per machine.
46 perMachine = { instances, machine, ... }: {
47     # Analog to 'perSystem' of flake-parts.
48     # For every machine of this service we will add exactly one nixosModule
49     nixosModule = { config, ... }: {
50         # Interaction examples what you could do here:
51         # - Get the name of this machine
52         # machine.name
53         #
54         # - Get all roles of this machine across all instances:
55         # machine.roles
56         #
57         # - Get the settings of a specific instance of a specific machine
58         # instances.foo.roles.peer.machines.jon.settings
59         #
60         # Globally enable something
61         networking.enable = true;
62     };
63 };
64 # ...
65 }

```

Using values from a NixOS machine inside the module

❖ Experimental Status

This feature is experimental and should be used with care.

Sometimes a settings value depends on something within a machines `config`.

Since the `interface` is defined completely machine-agnostic, this means values from a machine cannot be set in the traditional way.

The following example shows how to create a local instance of machine specific settings.

`someservice.nix`

```

1 {
2     # Maps over all instances and produces one result per instance.
3     perInstance = { instanceName, extendSettings, machine, roles, ... }:
4         nixosModule = { config, ... }:
5             let
6                 # Create new settings with
7                 # 'ipRanges' defaulting to 'config.network.ip.range' from th
8                 # This only works if there is no 'default' already.
9                 localSettings = extendSettings {
10                     ipRanges = lib.mkDefault config.network.ip.range;
11                 };
12             in
13             {
14                 # ...
15             };
16 };
17 }
```

⚠ Danger

`localSettings` are a local attribute that other machines cannot access, because calling `extendSettings` doesn't change the original `settings`. This means if a different machine tries to access, for example `roles.client.settings`, it will *NOT* contain changes made by `extendSettings`.

Exposing the changed settings to other machines would come with a huge performance penalty, that's why we don't want to offer it.

Passing `self` or `pkgs` to the module

Passing any dependencies in general must be done manually.

In general we found the following two best practices:

1. Using `lib.importApply`
2. Using a wrapper module

Both have pros and cons. After all using `importApply` is the easier one, but might be more limiting sometimes.

Using `importApply`

Using `importApply` is essentially the same as `import file` followed by a function-application; but preserves the error location.

Imagine your module looks like this

`messaging.nix`

```
1 { self }:
2 { ... }: # This line is optional
3 {
4     _class = "clan.service";
5     manifest.name = "messaging"
6     # ...
7 }
```

To import the module use `importApply`

`flake.nix`

```
1 # ...
2 outputs = inputs: flake-parts.lib.mkFlake { inherit inputs; } ({self, lib}
3     imports = [ inputs.clan-core.flakeModules.default ];
4     # ...
5     # Sometimes this attribute set is defined in clan.nix
6     clan = {
7         # Register the module
8         modules."@hsjobeki/messaging" = lib.importApply ./service-modules;
9     };
10 })
```

Using a wrapper module

messaging.nix

```
1 { config, ... }:
2 {
3     _class = "clan.service";
4     manifest.name = "messaging"
5     # ...
6     # config.myClan
7 }
```

Then wrap the module and forward the variable `self` from the outer context into the module

flake.nix

```
1 # ...
2 outputs = inputs: flake-parts.lib.mkFlake { inherit inputs; } ({self, lib}
3     imports = [ inputs.clan-core.flakeModules.default ];
4     # ...
5     # Sometimes this attribute set is defined in clan.nix
6     clan = {
7         # Register the module
8         modules."@hsjobeki/messaging" = {
9             # Create an option 'myClan' and assign it to 'self'
10            options.myClan = lib.mkOption {
11                default = self;
12            };
13            imports = [ ./service-modules/messaging.nix ];
14        }
15    };
16 })
```

The benefit of this approach is that downstream users can override the value of `myClan` by using `mkForce` or other priority modifiers.

Example: A machine-type service

Users often have different types of machines. These could be any classification you like, for example "servers" and "desktops". Having such distinctions, allows reusing parts of your

configuration that should be applied to a class of machines. Since this is such a common pattern, here is how to write such a service.

For this example we have two roles: `server` and `desktop`. Additionally, we can use the `perMachine` section to add configuration to all machines regardless of their type.

`machine-type.nix`

```
1 {
2   _class = "clan.service";
3   manifest.name = "machine-type";
4
5   roles.server.perInstance.nixosModule = ./server.nix;
6   roles.desktop.perInstance.nixosModule = ./desktop.nix;
7
8   perMachine.nixosModule = {
9     # Configuration for all machines (any type)
10  };
11 }
12
```

In the inventory we then assign machines to a type, e.g. by using tags

`flake.nix`

```
1 instances.machine-type = {
2   module.input = "self";
3   module.name = "@pinpox/machine-type";
4   roles.desktop.tags.desktop = { };
5   roles.server.tags.server = { };
6 };
```

Sharing Data Between Machines with Exports

Services often need to share configuration data between machines. For example, clients need to know the server's IP address, or peers in a mesh network need to discover each other.

Exports provide a standardized way to share structured data between service components.

 See the complete guide: [Service Exports](#)

Further Reading

- [Service Exports Guide](#) - How to share data between services
- [Reference Documentation for Service Authors](#)
- [Migration Guide from ClanModules to ClanServices](#)
- [Decision that lead to ClanServices](#)
- [Testing Guide for Services with Vars](#)

← Previous

Introduction to Services

Next →
Exports

Need help? Reach out to [our community](#).